



SigmaStar Display

用户开发指南



© 2020 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.



{SigmaStar Display}

{Smart Display}

{SSD201 + 1.0}

REVISION HISTORY

Revision No.	Description	Date
0.1	Initial release	06/21/2019
0.2	Update	31/10/2019

TABLE OF CONTENTS

REVISION HISTORY	i
TABLE OF CONTENTS	ii
1. 熟悉 SDK 组成	1
1.1. 下载 SDK	1
1.2. 编译环境搭建	1
1.2.1 安装 Linux 服务器	2
1.3. 编译 SDK	3
1.3.1 解压 code	4
1.3.2 编译 boot	4
1.3.3 编译 kernel	4
1.3.4 编译 SDK(ALKAID)	4
1.4. 烧写	5
1.4.1 Burning code by uboot	5
1.4.2 Burning uboot by ISP Tool	6
1.5. 如何自定义 Flash 分区	错误!未定义书签。
1.5.1 Spinor 分区信息调整	错误!未定义书签。
1.6. 内存使用详解	12
1.7. MMAP 调整方法	14
1.7.1 mmap.ini 说明	14
1.7.2 调整 mmap.ini 中的 Layout	14
1.8. 注意事项	15

1. 熟悉 SDK 组成

1.1. 下载 SDK

从 SStar 提供的 ftp 可以 down 到如下资源：

 boot-TAKOYAKI-ALPHA001-0606.tar.gz
 gcc-arm-8.2-2018.08-x86_64-arm-linux-gnueabi.tar.gz
 kernel-TAKOYAKI-ALPHA001-0606.tar.gz
 project-TAKOYAKI-ALPHA001-0606.tar.gz
 sdk-TAKOYAKI-ALPHA001-0606.tar.gz

Figure 1: 图 1-1 SDK 包

boot: Uboot 源码

kernel: kernel 源码，版本：4.9.84

project: 编译制作 image 的部分，包含 release 的头文件以及 lib 库

sdk: 测试代码部分

gcc-xxx: toolchain 部分，glibc 版本提供的是 8.2.1 64bit

1.2. 编译环境搭建

通常我们会以交叉编译的方式进行开发和调试，即“宿主机+目标机”的形式。而宿主机和目标机的连接我们一般采用串口连接或网络连接，如下图所示：



Figure 2: 图 1-2 调试连接方式

注：我们提供的 debug tool 作用除了可以 debug 外，还可以通过 PC 工具读寄存器和烧写空 flash。

如果是仅调试可以使用串口设备正确对接调试线序即可(con3)，线序请和原理图对齐。

如果需要读 Register 或者烧录裸 Flash 的时候，需要 Debug Tool。

tool 根据需求量自行购买 <https://item.taobao.com/item.htm?id=15437784110>

1.2.1 安装 Linux 服务器

建议使用 Ubuntu 16.04。

由于 sdk 的编译需要用到一些 lib&tool，这里就将安装流程介绍一下。

1.2.1.1. 安装 ubuntu 16.04 server

- 1、下载 1604 的 server 版：<http://www.ubuntu.org.cn/download/server>，选择下载 1604 server 版
- 2、安装 VMware Workstation
- 3、选择 1604 的 iso，装载然后安装即可
- 4、简易安装直接点击完成，系统会自动进行安装
- 5、安装 VMtools，用于拷贝将 windows 目录，共享到 ubuntu，参考：
<https://www.cnblogs.com/huangjianxin/p/6343881.html>

ubuntu 至此安装成功，接下来安装开发需要的一些环境。

1.2.1.2. 安装编译需要的 lib & tool

接下来安装相应的编译工具等：

- 1、新装 ubuntu，首先要设置 root 密码，进入系统设置 root 密码：`# sudo passwd`，连续输入密码即可，提示成功即可
- 2、添加用户，在创建虚拟机时会提示输入 user count，但是如果要使用 beal.wu 等类似账号，在创建 ubuntu 的时候是无法输入的（会提示格式错误），系统会提示账户非法，但是由于个人需要使用 beal.wu，那么需要进入系统后重新创建用户，`# sudo adduser beal.wu --force` 来创建用户，此时会提示输入 account 的一些信息，一直回车即可。
- 3、此时账户已经建立，那么需要安装相关的工具以及编译环境，执行 `# sudo apt-get install libncurses5-dev libncursesw5-dev`，这时候会提示 beal.wu 无法执行 sudoers，需要做如下修改：

```
sudo chmod u+w /etc/sudoers
```

```
vi /etc/sudoers;
```

```
参考      root      ALL=(ALL:ALL) ALL,
```

```
添加一行  beal.wu  ALL=(ALL:ALL) ALL
```

```
sudo chmod u-w /etc/sudoers
```

再执行 `sudo` 操作即可安装了，建议在安装完后执行 `#sudo apt-get update` 来更新源

- 4、安装 samba，用于共享 linux 文件到 windows

```
# sudo apt-get install samba samba-common
```

```
# sudo smbpasswd -a beal.wu （添加 samba 账户，用于 windows 访问 linuxsamba 目录）
```

修改/etc/samba/smb.conf

```
[beal.wu]
```

```
path=/home/beal.wu
```

```
public=yes
```

```
writable=yes
```

```
valid users=beal.wu
```

```
available=yes
```

```
browseable=yes
```

```
guest ok = yes
```

这样 samba 已经配置好了，重启 samba 服务即可，测试从 windows 访问 samba: \\192.168.1.11 (your ubuntu ip)

5、安装 ssh，用于登陆调试

sudo apt-get install openssh-server; 安装完了即可使用

6、针对 SDK 编译需要安装一些 tool，否则会编译失败

```
# sudo apt-get install libc6-dev-i386
# sudo apt-get install lib32z1 lib32ncurses5
# sudo apt-get install libuuid1:i386
# sudo apt-get install cmake
# sudo apt-get install libncurses5-dev libncursesw5-dev
# sudo apt install bc
# sudo apt-get install xz-utils
# sudo apt-get install automake
# sudo apt-get install libtool
# sudo apt-get install libevdev-dev
# sudo apt-get install pkg-config
```

如上这些需要安装的 tool && lib 是必须的，因为编译过程中会用到相关内容。这边就不一一说明是哪个错误，可以尝试先不安装编译来查看相关错误，为了省事，在编译之前一次性安装即可。

7、如果默认 sh 不是 bash，需要将 sh 改成 bash:

Sudo rm /bin/sh

Sudo ln -s /bin/bash /bin/sh

8、安装 toolchain

拷贝 “gcc-arm-8.2-2018.08-x86_64-arm-linux-gnueabi.tar.gz” 到/opt/ 目录自行定义

解压 toolchain

将 toolchain 设置到环境变量中去，省得每次需要手动 export:

在/etc/profile 添加: export PATH=/opt/gcc-arm-8.2-2018.08-x86_64-arm-linux-gnueabi/bin:\$PATH

9、到这里，就是执行编译了

1.2.1.3. 安装交叉编译工具

我们提供编译工具链 gcc-arm-8.2-2018.08-x86_64-arm-linux-gnueabi 用于编译 glibc 版本的。

Glibc: 2.28

```
beal.wu@sigmastar:~/i2m/sdk$ /opt/ToolChain/gcc-arm-8.2-2018.08-x86_64-arm-linux-gnueabi/bin/arm-linux-gnueabi-gc
Using built-in specs.
COLLECT_GCC=/opt/ToolChain/gcc-arm-8.2-2018.08-x86_64-arm-linux-gnueabi/bin/arm-linux-gnueabi-gcc
COLLECT_LTO_WRAPPER=/opt/ToolChain/gcc-arm-8.2-2018.08-x86_64-arm-linux-gnueabi/bin/../libexec/gcc/arm-linux-gnueabi
Target: arm-linux-gnueabi
Configured with: /tmp/dgboter/bbs/bc-b1-2-11--rhe6x86_64/buildbot/rhe6x86_64--arm-linux-gnueabi/build/src/gcc/config
ith-sysroot=/arm-linux-gnueabi/libc --with-build-sysroot=/tmp/dgboter/bbs/bc-b1-2-11--rhe6x86_64/buildbot/rhe6x86_64
ueabi/install/arm-linux-gnueabi/libc --enable-gnu-indirect-function --enable-shared --disable-libssp --disable-li
ing=release --enable-languages=c,c++,fortran --with-gmp=/tmp/dgboter/bbs/bc-b1-2-11--rhe6x86_64/buildbot/rhe6x86_64--a
bihf/host-tools --with-mpfr=/tmp/dgboter/bbs/bc-b1-2-11--rhe6x86_64/buildbot/rhe6x86_64--arm-linux-gnueabi/build/bui
tmp/dgboter/bbs/bc-b1-2-11--rhe6x86_64/buildbot/rhe6x86_64--arm-linux-gnueabi/build/build-arm-linux-gnueabi/host-t
he6x86_64/buildbot/rhe6x86_64--arm-linux-gnueabi/build/build-arm-linux-gnueabi/host-tools --with-arch=armv7-a --wi
7-a --with-pkgversion='GNU Toolchain for the A-profile Architecture 8.2-2018-08 (arm-rel-8.23)'
Thread model: posix
gcc version 8.2.1 20180802 (GNU Toolchain for the A-profile Architecture 8.2-2018-08 (arm-rel-8.23))
beal.wu@sigmastar:~/i2m/sdk$
```

1.3. 编译 SDK

SSD20x 支持 nor flash 和 spi nand flash 两种启动方式，因此在 SDK 中编译有所区分，通过不同的配置文件来实现，SDK 中包含了 uboot 和 kernel 的 sourcecode，通过 project 中脚本打包可生成可烧录的 images。

Spinor/Spinand 都可以正常工作，Display demo board 搭载的是 Spinor flash，客户可以自行 layout，如果在 SSD20x 的 Display demo board 板需要换成 Spinand 调试，请联系我们硬件。

1.3.1 解压 code

从 FTP 获取各自的 SDK 开发包，获取到 sourcecode，格式如：（截图名称仅供参考）

```
tar zxvf boot-TAKOYAKI-ALPHA001-0606.tar.gz
```

...

```
tar zxvf sdk-TAKOYAKI-ALPHA001-0606.tar.gz
```

1.3.2 编译 boot

- **SPI-NOR package**

```
#declare -x ARCH="arm"
#declare -x CROSS_COMPILE="arm-linux-gnueabihf-"
#make infinity2m_defconfig; //spinor
#make infinity2m_spinand_defconfig; //Spinand
#make menuconfig; //modify && save
#make clean;
#make
```

- **Get image**

```
# cp u-boot.xz.img.bin ${ your_release_path } //spinor
# cp u-boot_spinand.xz.img.bin ${ your_release_path } //Spinand
```

1.3.3 编译 kernel

- **Kernel**

CHIP	Glibc compiler	Ulibc compiler	Kernel make config
SSD20x spinor	arm-linux-gnueabihf-	NA	infinity2m_ssc011a_s01a_minigui_defconfig
SSD20x spinand	arm-linux-gnueabihf-	NA	infinity2m_spinand_ssc011a_s01a_minigui_defconfig

注：请参考表格和你的芯片版本做对应的编译，Ulibc 第一版 SDK 暂不支持，后续完善。

```
# declare -x ARCH="arm"
# declare -x CROSS_COMPILE=" arm-linux-gnueabihf-"
# make infinity2m_ssc011a_s01a_minigui_defconfig //SSD20x spinor
# make infinity2m_spinand_ssc011a_s01a_minigui_defconfig //SSD20x Spinand
# make menuconfig; //modify && save
# make clean;
# make
```

- **Get image**

```
# cp arch/arm/boot/uImage.xz ${ your_release_path }
```

注意：如果要更新 uboot、kernel，需要先编译 project，然后将 uboot 和 kernel 的 bin 拷贝至 /yourpath/xxx/image/output/image 中，kernel 是 ulmage.xz -> kernel

1.3.4 编译 SDK(ALKOID)

- **Build Sdk package**

CHIP	Glibc	Ulibc
QFN128 64M: SSD201 Nor	nor.glibc-squashfs.011a.64	
QFN128 64M: SSD201 Nand	spinand.glibc.011a.64	

QFN128 128M: SSD202D Nor	<code>nor.glibc-squashfs.011a.128</code>	
QFN128 128M: SSD202D Nand	<code>spinand.glibc.011a.128</code>	

注：请参考表格和你的芯片版本做对应的编译。

```
# cd ${your path}/project
```

8.2.1: s

```
# ./setup_config.sh ./configs/nvr/i2m/8.2.1/nor.glibc-squashfs.011a.64 //SSD20x Spinor
```

```
# ./setup_config.sh ./configs/nvr/i2m/8.2.1/spinand.glibc.011a.64 //SSD20x Spinand
```

```
# make clean; make image
```

- **Get image**

```
# cd ${Alkaid}/project/image/output/images
```

1.4. 烧写

1.4.1 Burning code by uboot

- **Run tftp (FTP server) on PC**

Step1. 使用 tftp 工具指向 image path: SDK\project\image\output\images\, 并选择正确的网卡。

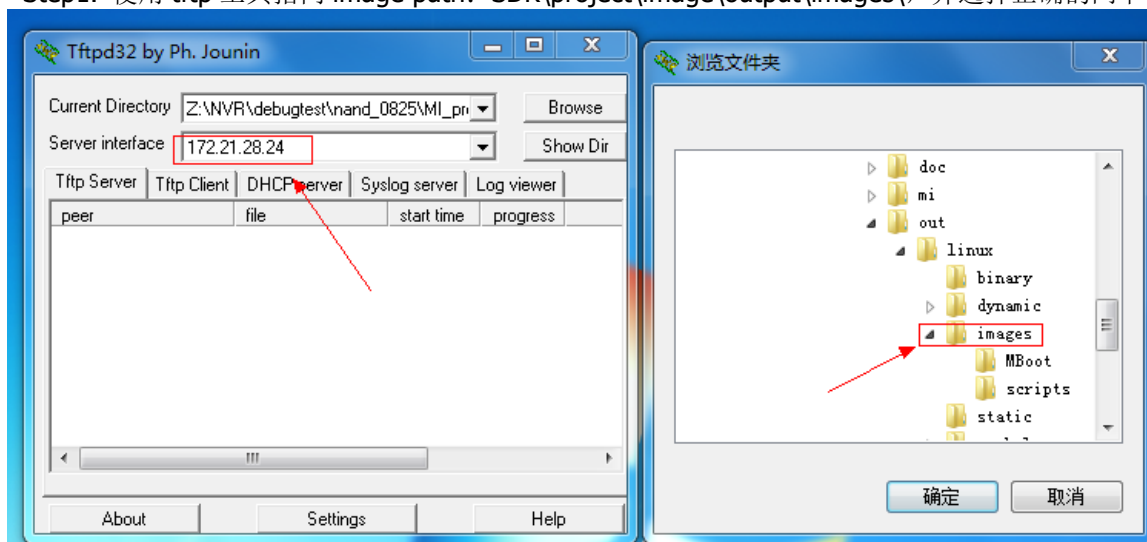


Figure 5: 图 1-2

Step 2. 连接板子的网口到 PC 端，连接 debug 串口工具到 PC 端，并检查连接的正确性。

- **Run tftp (FTP Client) on Demo Board**

Step 1. 板子开机，长按回车，进入 bootloader command line. Mstar 的 boot loader.

■ 首次烧录请设置 IP: (除非 ip 设置变更或者更换 flash)

```
# setenv gatewayip 192.168.1.1
```

```
# setenv ipaddr 192.168.1.127 //设定 FTP Client (EVB 板子)使用的 IP
```

```
# setenv netmask 255.255.255.0
```

```
# setenv serverip 192.168.1.100 //设定 FTP server (PC) 的 IP
```

```
# saveenv
```

注：

1. 为了保证烧录顺利，请保证 PC 和开发板处于同一网段。

2. 请采用静态方式固定分配 ip。防止烧录时 ip 地址跳变。

3. 您也可以使用独立网卡使 PC 端直连开发板，固定该网卡的内网 ip 地址，并按上述方法设定开发板。

■ 在 UBOOT console 下执行以下指令即可自动透过 ethernet 烧录。

estar (OR: estar auto_update.txt, 也可以烧录其中某一个分区：)

1.4.2 Burning uboot by ISP Tool

烧录裸 flash

本方式适用于空机烧录，或者 uboot 已经损坏导致无法通过 uboot 升级的场合。

1.4.2.1. SPI-NOR-Flash

● Default Partition layout

No	range	size
IPL	0x00000000-0x00010000	64KB
IPL_CUST	0x00010000-0x00020000	64KB
MXPT	0x00020000-0x00030000	64KB
UBOOT	0x00030000-0x0004F000	124KB
UBOOT_ENV	0x0004F000-0x00050000	4KB
BOOT	0x00000000-0x00050000	256KB
KERNEL	0x00050000-0x00250000	2048KB
rootfs	0x00250000-0x00550000	3072KB
nvrservice	0x00550000-0x009D0000	4800KB
customer	0x009D0000-0x00F00000	5300KB

Table 1: 表 1-1

表格仅供参考，分区信息请按这个为准：board/i2m/boot/nor/partition/tool/mxp_gen.c 中如下代码段为分区信息。

```
i=0;
out_record("IPL", MXP_PART_TYPE_NORMAL, 0x00000000, 0x00010000, MXP_PART_STATUS_EMPTY, i++); //don't modify
out_record("IPL_CUST", MXP_PART_TYPE_NORMAL, 0x00010000, 0x00020000, MXP_PART_STATUS_EMPTY, i++); //don't modify
out_record("KEY_CUST", MXP_PART_TYPE_NORMAL, 0x00020000, 0x00030000, MXP_PART_STATUS_EMPTY, i++); //0x3000,0x20000,0x40000
out_record("MXPT", MXP_PART_TYPE_NORMAL, 0x00030000, 0x0004F000, MXP_PART_STATUS_EMPTY, i++);
out_record("UBOOT", MXP_PART_TYPE_NORMAL, 0x0004F000, 0x00050000, MXP_PART_STATUS_EMPTY, i++);
out_record("UBOOT_ENV", MXP_PART_TYPE_NORMAL, 0x00050000, 0x00055000, MXP_PART_STATUS_EMPTY, i++);
out_record("BOOT", (MXP_PART_TYPE_NORMAL|MXP_PART_TYPE_MTD), 0x00055000, 0x00250000, MXP_PART_STATUS_EMPTY, i++);
out_record("KERNEL", (MXP_PART_TYPE_NORMAL|MXP_PART_TYPE_MTD), 0x00250000, 0x00550000, MXP_PART_STATUS_EMPTY, i++);
out_record("rootfs", (MXP_PART_TYPE_NORMAL|MXP_PART_TYPE_MTD), 0x00550000, 0x009D0000, MXP_PART_STATUS_EMPTY, i++);
out_record("nvrservice", (MXP_PART_TYPE_NORMAL|MXP_PART_TYPE_MTD), 0x009D0000, 0x00F00000, MXP_PART_STATUS_EMPTY, i++);
out_record("customer", (MXP_PART_TYPE_NORMAL|MXP_PART_TYPE_MTD), 0x00F00000, 0x00F00000, MXP_PART_STATUS_EMPTY, i++);
```

当前 flash 中已经烧录的分区信息可以在 uboot 下使用 mxp t.list 查看

SigmaStar # mxp t.list

● Burning code by ISP tool

	offset	Binary 放置目录
IPL.bin	0x0000	\${ALKALID}\project\image\output\images\IPL.bin
IPL_CUST.bin	0x10000	\${ALKALID}\project\image\output\images\IPL_CUST.bin
MXPT_SF.bin	0x20000	\${ALKALID}\project\image\output\images\MXPT_SF.bin
u-boot.xz.img.bin	0x30000	\${ALKALID}\project\image\output\images\u-boot.xz.img.bin

Table 2: 表 1-2

● Burning Steps

■ Step1. 执行 ISP tool。并且关闭 UART terminal，否则无法正常‘Connect’。

如果是在 uboot 下则输入“debug”关闭，在系统下输入“11111”。

注：一定要关闭串口连接。

- Step2. 选择 SPI tab, 点击'More' 并且选择类型为'SPI'。

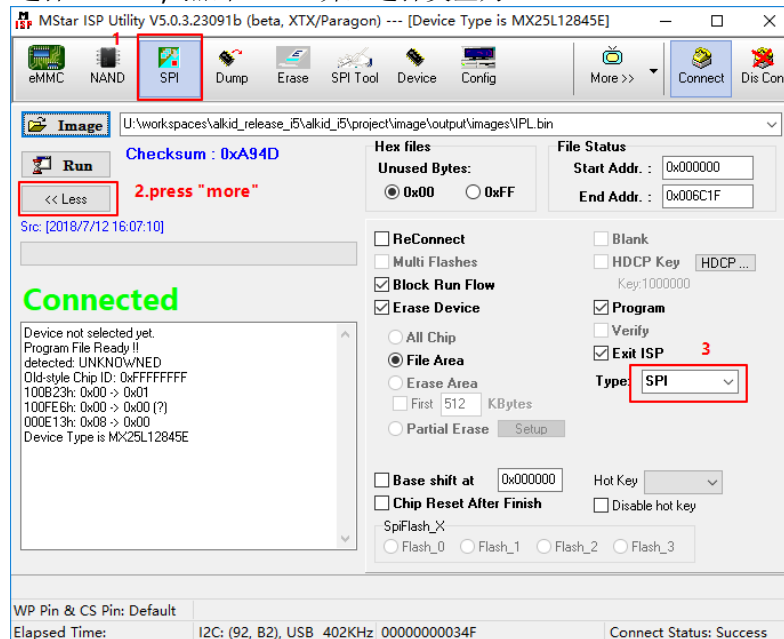


Figure 3: 图 1-3

- Step3. 加载烧录文件并点击'Connect'。

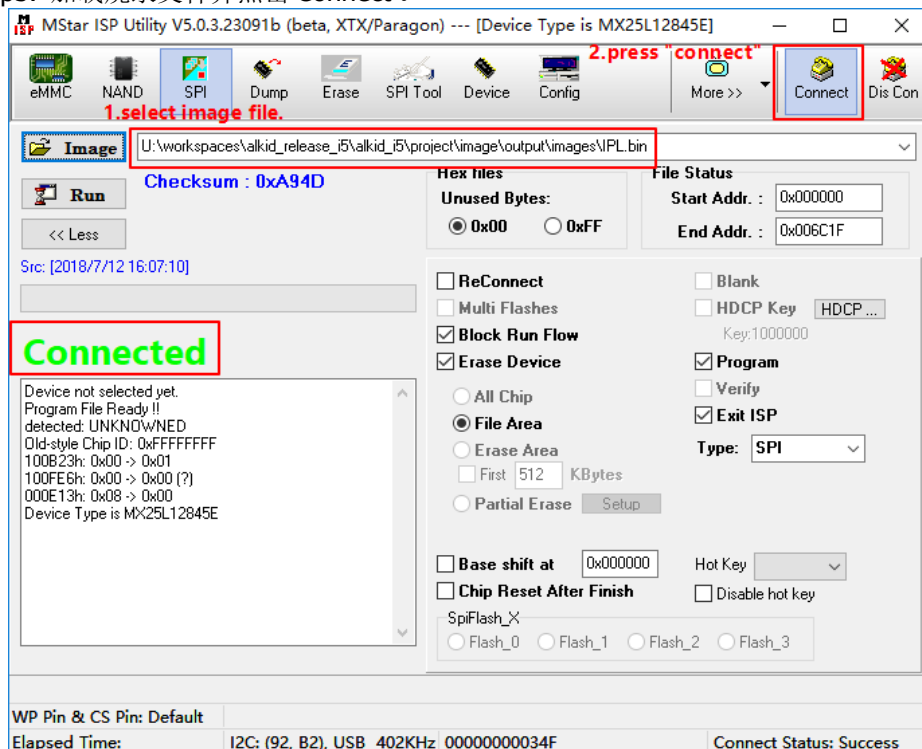


Figure 4: 图 1-4

- Step4. 加载 image "IPL.bin",并点击'Run'。

注: 需要勾选'erase file area'。

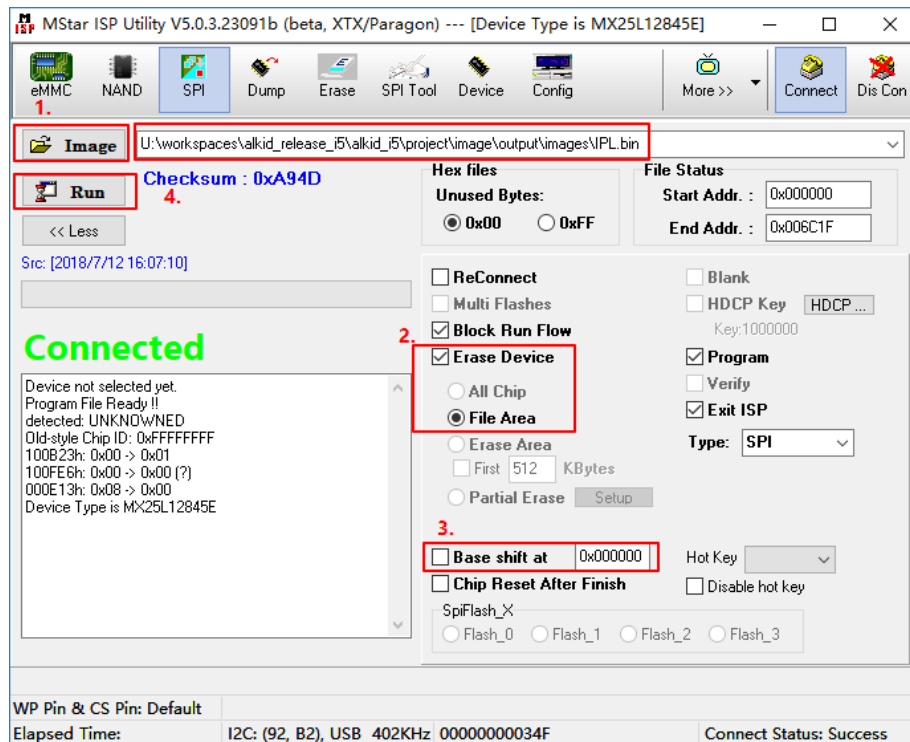


Figure 5: 图 1-5

- Step5. 加载 image “IPL_CUST.bin”，取消“Erase Device”选项。设置“Base shift”at 0x10000。
注:可能随着版本变化，Base shift 的地址以表格 1-2 数据为准。

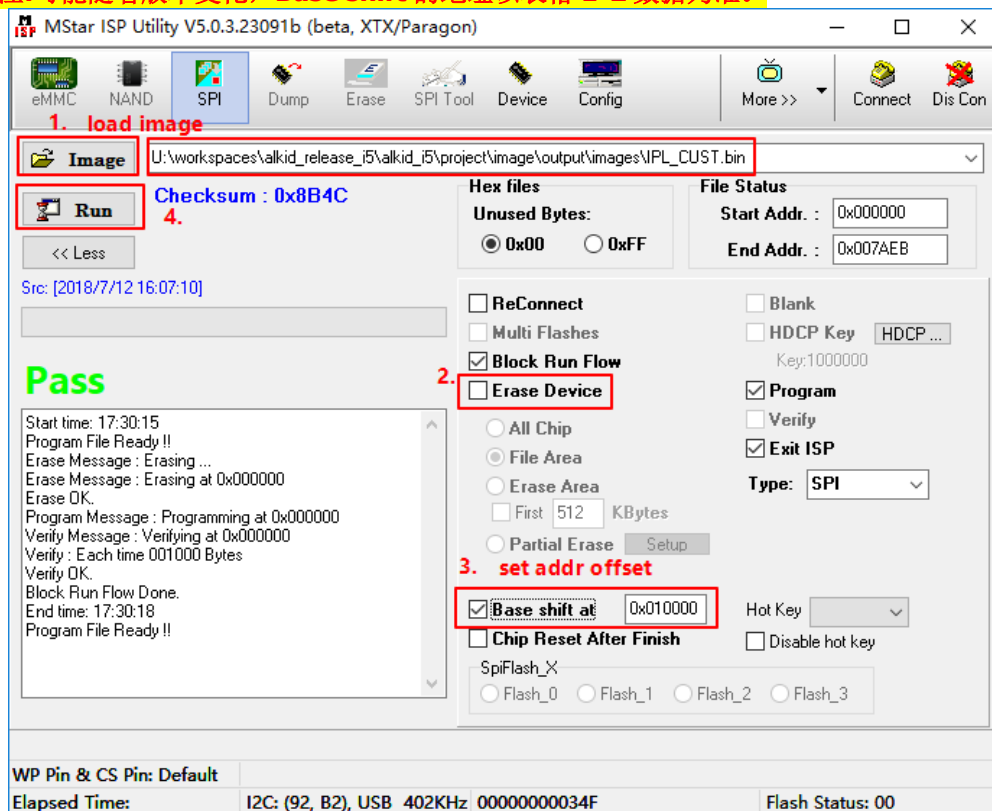


Figure 6: 图 1-6

- Step6. 加载 image “MXP_SF.bin”，设置“Base shift”at 0x20000。

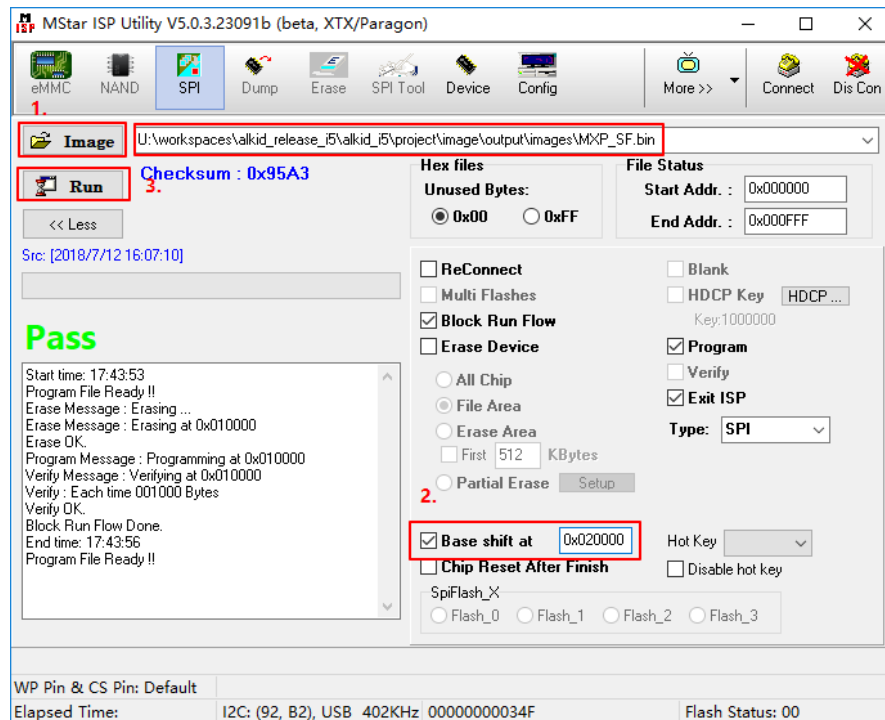


Figure 7: 图 1-7

- Step7. 加载 image “u-boot.xz.img.bin”，设置‘Base shift’at 0x30000。

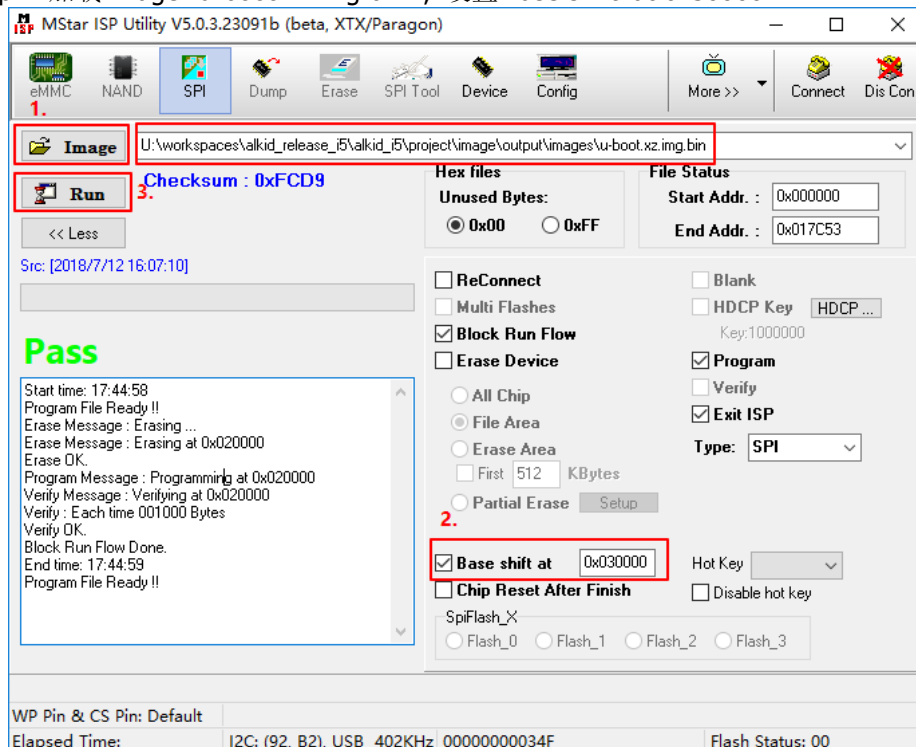


Figure 8: 图 1-8

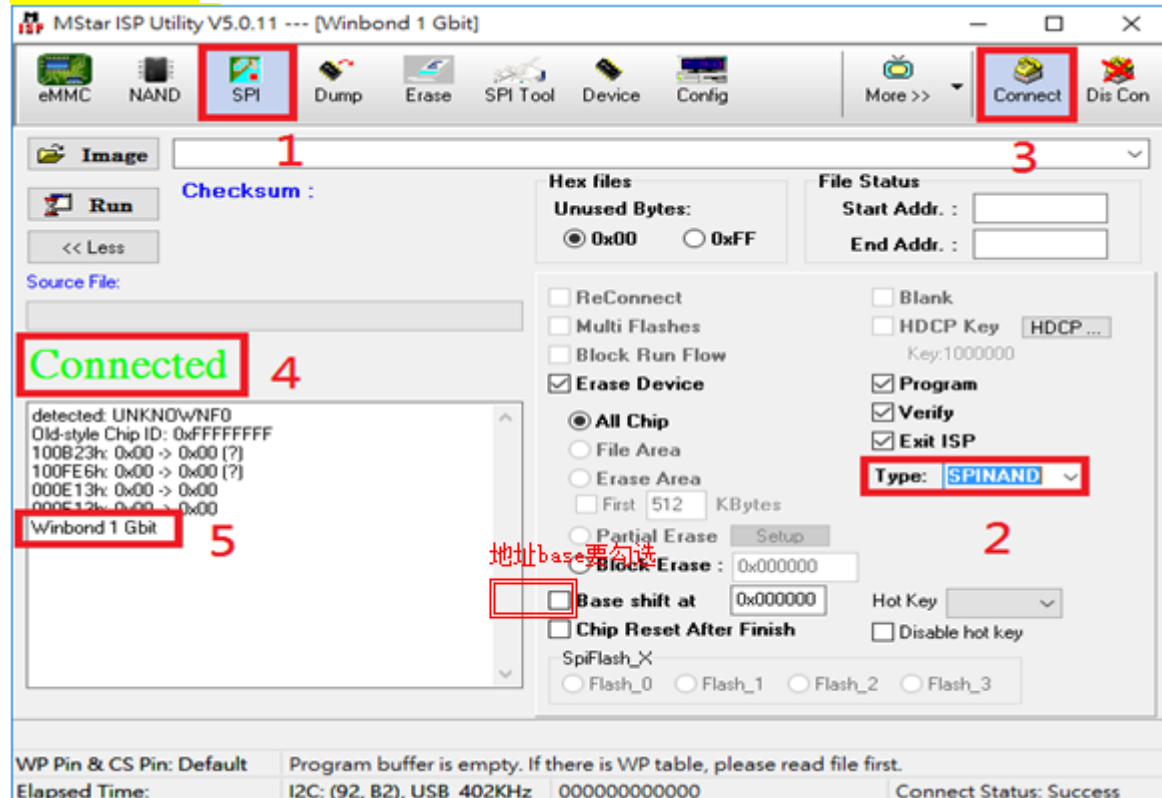
- Step8. 重启 **Demo Board** 板子，关闭工具，接下来通过网络可以烧录参考 1.4.1 到这里，应该可以进入 Uboot 了，进入 Uboot 使用 TFTP 烧录方法烧录剩下的部分。

1.4.2.2. Spinand flash

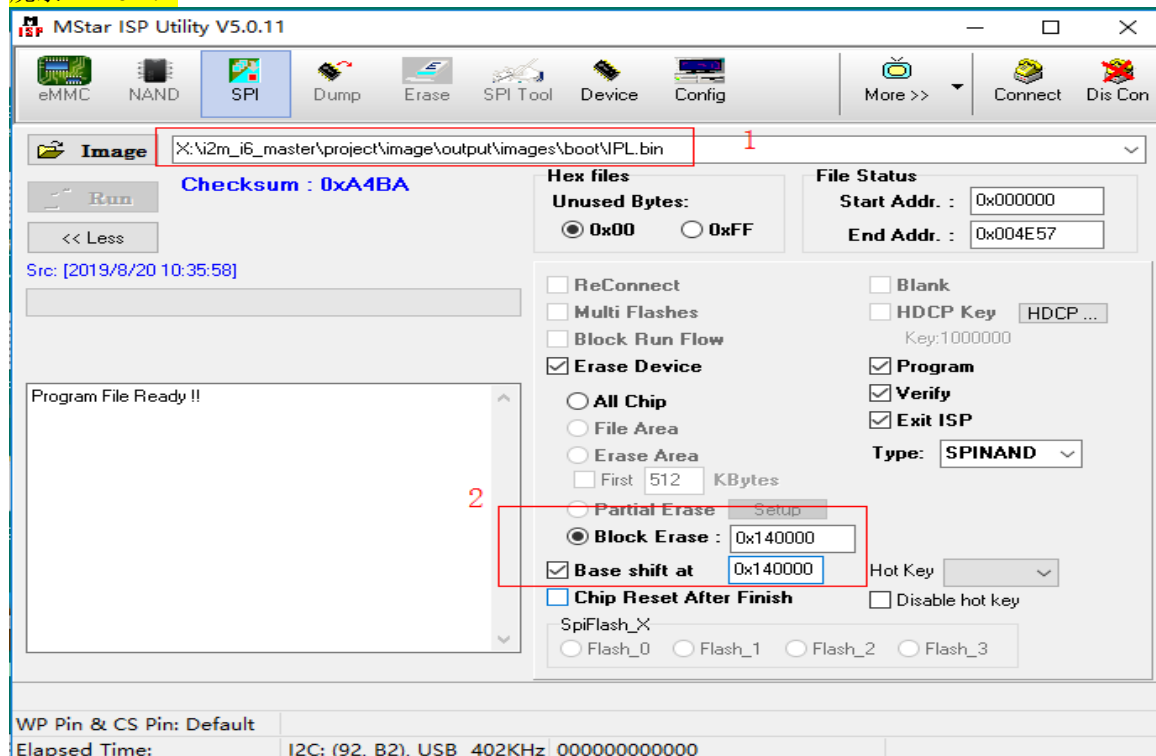
板子连接等配置和 Spinor 一样，区别是在选 Flash type 的时候需要选择 SPINAND。

烧录选择需要选择 spinand 然后连接：

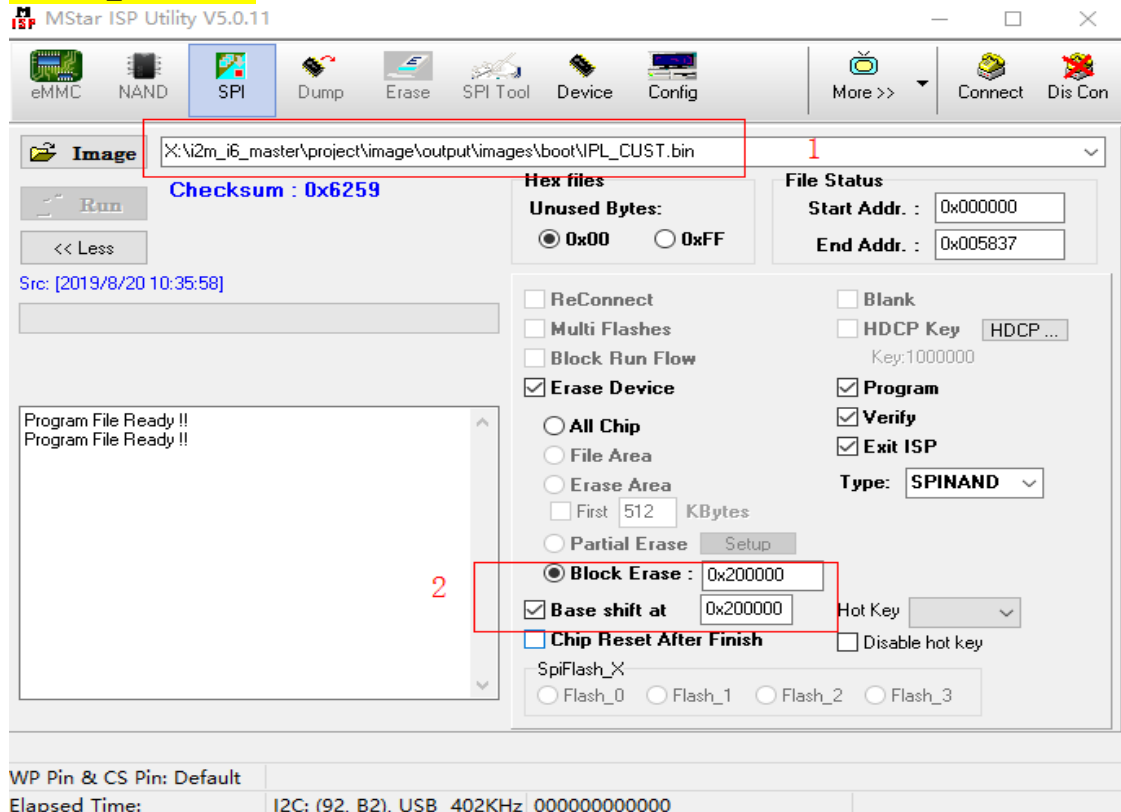
烧录 GCIS.bin:



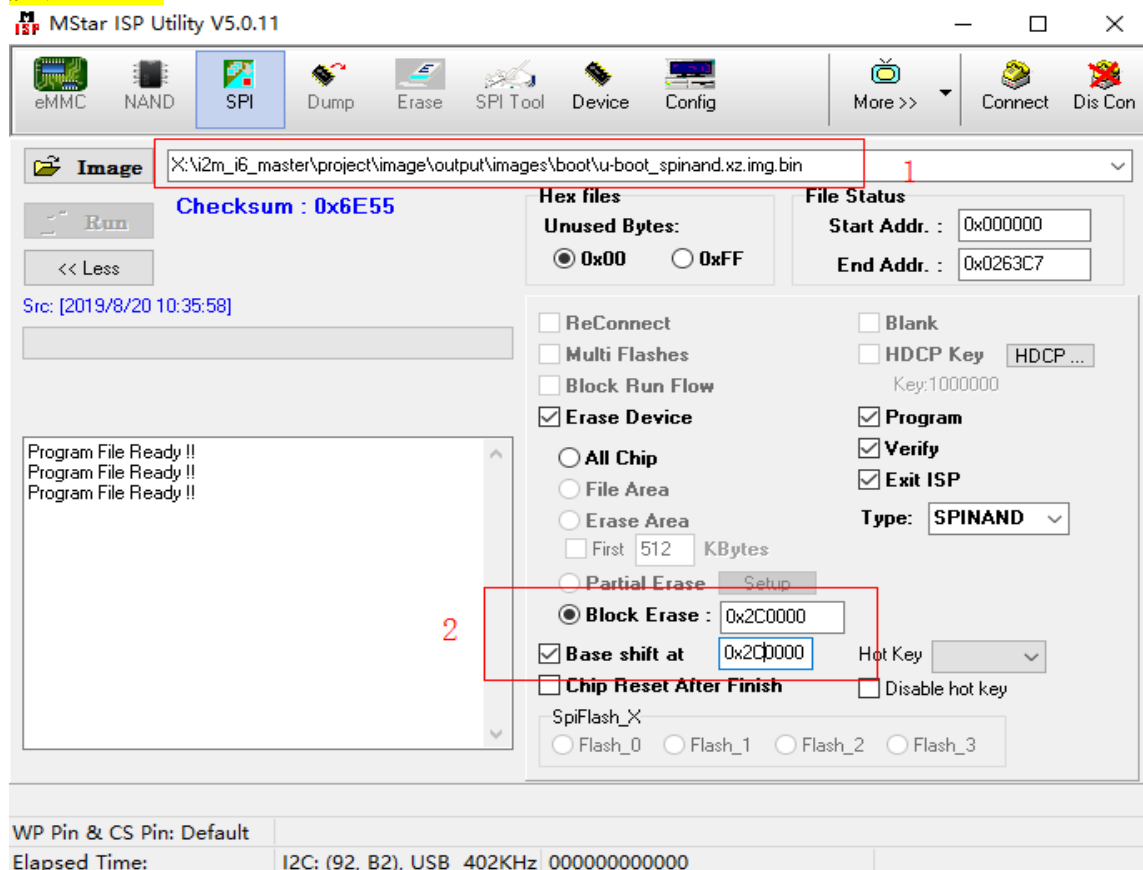
烧录 IPL.bin:



烧录 IPL_CUST.bin:



烧录 uboot:



烧录 spinand 的方法和烧录 spinor 一样，只是地址以及文件稍有差别

	Erase Device	offset	Image 所在目錄
GCIS.bin	All Chip	0x00000	yourPath/image
IP.bin	File Area	0x140000	yourPath/image
IPL_CUST.bin	File Area	0x200000	yourPath/image
u-boot_spinand.xz.img.bin	File Area	0x2C0000	yourPath/image

依次烧录上述文件后，就可以进入 Uboot，然后可以使用网络烧录其他 images 了。

注：烧录时要求停掉串口，并关闭串口工具

Uboot 下停串口：输入 debug

Kernel 下停串口：输入 11111

空 flash 时，如何确定 cpu 是 ok 的，将串口 buad 设置 38400 即可看到打印，而非乱码

进入 uboot 执行：estar 可以执行 tftp 升级了。

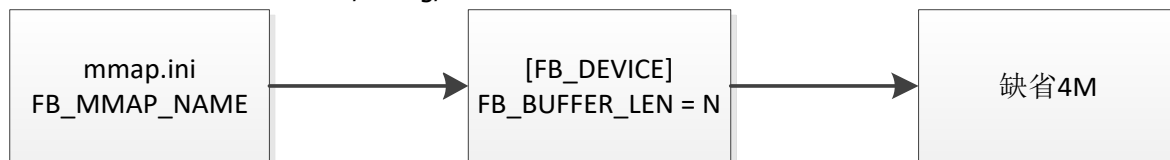
1.5. 内存使用详解

下表描述的是内存分配的情况：内存等于（LX+Layout）两部分

	组成	SIZE	用途	是否可配置
LX	MMA	21M (可能已经修改)	预留分配一块内存, 供 SDK 中的模块分配, 比如 VDEC、VENC 等, 由于这些需要连续的物理内存, 所以我们会预留出来, 并交由 MI_SYS 管理	可以, 根据客户的需求修改, 正常如果仅要 1080P 解码显示, 大概 9M 即可
	Linux mem	44M	LX 除了预留的 mma, 剩下的内存由 linux 管理, 缺省大概从 /proc/meminfo 中可以看到是 40M, 原因是 kernel 的 text 段会预留出一段内存, 如下大约 4M (和 kernel config 有关, kernel 越大预留会越多, 所以需要裁剪无关的配置) / # cat /sys/kernel/debug/memblock/reserved 0: 0x20004000..0x20007fff 1: 0x20008240..0x203cfeb7	可以
LAYOUT	AI、AO 等	1M	预留给一些必须的小块内存, 由于此部分也需要物理连续, 为了不让其去切碎大块内存, 所以给 layout 出来, 固定其物理地址	可以, 需要 1M 必须预留出来的内存
total		64M		
Mem layout 配置	/project/board/i2m/mmap/MMAP_I2M_64M.h 使用 SCA.exe 工具快速调整			
	/project/configs/nvr/i2m/8.2.1/nor.glibc-squashfs.011a.64 中 sz=0x1500000			
	cma 配置, cma 内存是在 meminfo total 中的, 设置方法可以在 bootargs 中添加 cma=2M			

之前有 FB 的内存也是 Layout 在 mmap.ini 中的, 但是由于灵活性以及屏幕分辨率变化太多, 需要经常重新 layout mmap.ini, 所以现在将 FB 的内存分配配置为动态从 mma 分配, 不过还是兼容了之前的分配方式, 优先从 mmap.ini

查找 FB 内存是否有配置, 然后查看 /config/fbdev.ini, 最后才会缺省按 fbdev 驱动分配。



1.6. MMAP 调整方法

1.6.1 mmap.ini 说明

芯片的内置 DDR2 容量为 64M，我们使用 mmap.ini 来作为内存的配置文件，其中包含被预分配出去的内存，这部分内存 linux 系统是在 linux 的管理中是看不到。

编译生成的 image 中/config/mmap.ini 来自/project/board/i2m/mmap，具体是使用的 project 中哪个头文件，则需要查看/project/configs/nvr/i2m/8.2.1/nor.glibc-squashfs.011a.64 中：

如 MMAP = MMAP_I2M_64M.h

如果是 SSD202 则需要修改为 MMAP = MMAP_I2M_128M.h

Linux 会解析 mmap.ini 来初始化内存部分，包括 mmap 中 layout 的 ai、ao 等

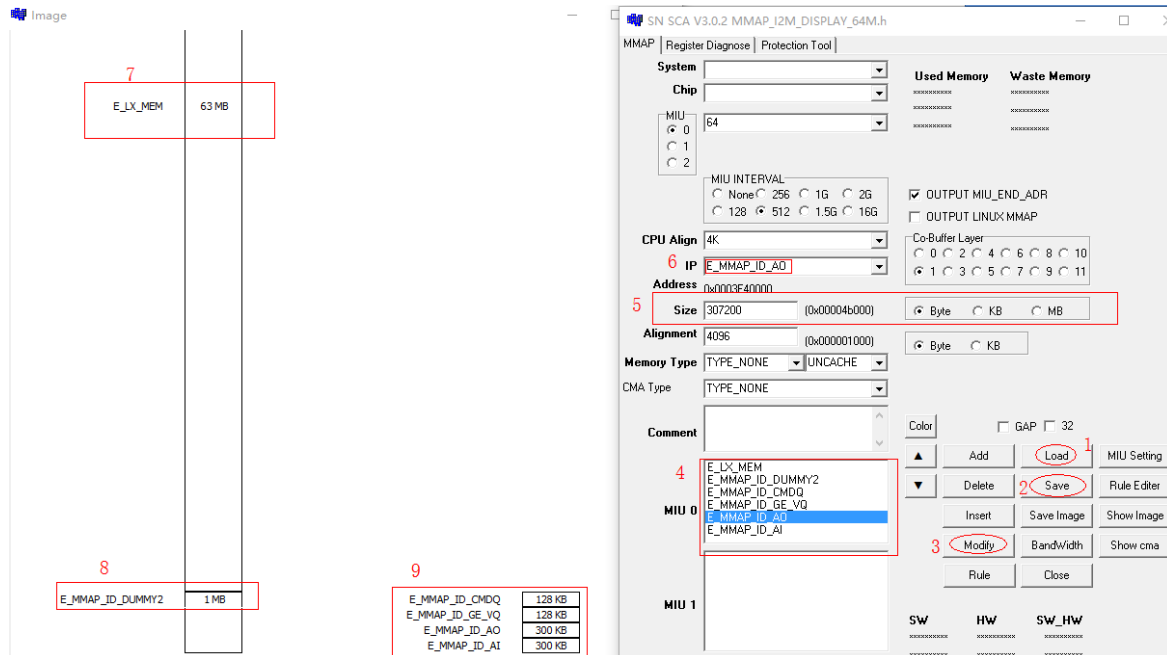
sz 预留的内存可以从 # cat /sys/kernel/debug/memblock/reserved

```
/ # cat /sys/kernel/debug/memblock/reserved
0: 0x20004000..0x20007fff
1: 0x20008240..0x20469637
2: 0x22000000..0x221fffff
3: 0x22265000..0x2228cfff
4: 0x2228f9c0..0x222fefff
5: 0x222ff440..0x222ff87b
6: 0x222ff880..0x222ff8bb
7: 0x222ff8c0..0x222ff937
8: 0x222ff940..0x222ff947
9: 0x222ff980..0x222ff987
10: 0x222ff9c0..0x222ff9c3
11: 0x222ffa00..0x222ffa03
12: 0x222ffa40..0x222ffba4
13: 0x222ffbc0..0x222ffd24
14: 0x222ffd40..0x222ffea4
15: 0x222ffeac..0x222ffec6
16: 0x222ffec8..0x222ffee2
17: 0x222ffee4..0x222ffefe
18: 0x222fff00..0x222fff1f
19: 0x222fff24..0x222fff3e
20: 0x222fff40..0x222fff4b
21: 0x222fff60..0x237fffff
```

1.6.2 调整 mmap.ini 中的 Layout

打开 SCA.exe 软件，load 配置文件，如下图：

- 1、点击 load 对应配置文件，如：/project/board/i2m/mmap/ MMAP_I2M_64M.h
- 2、修改完后点击“save”将当前修改保存
- 3、选中对应的 layout 块，然后修改对其 size 进行修改，点击“modify”完成对选中块的编辑
- 4、当前 mmap.ini 中已经编辑的 layout 内存块
- 5、Size 修改的区域，可以选择 BYTE、KB、MB，然后在输入框输入对应的容量即可
- 6、当前选择 layout 块的名称，也可以编辑，如新增一个 layout 块即可在此区域输入自定义名称
- 7、当前 mmap.ini 配置中 LX 的长度
- 8、当前 mmap.ini 配置中 DUMMY 的长度，dummy 包含了较多的小块内存
- 9、不足 1M 的内存，依附在 dummy layout 块下面



1.7. 注意事项

无